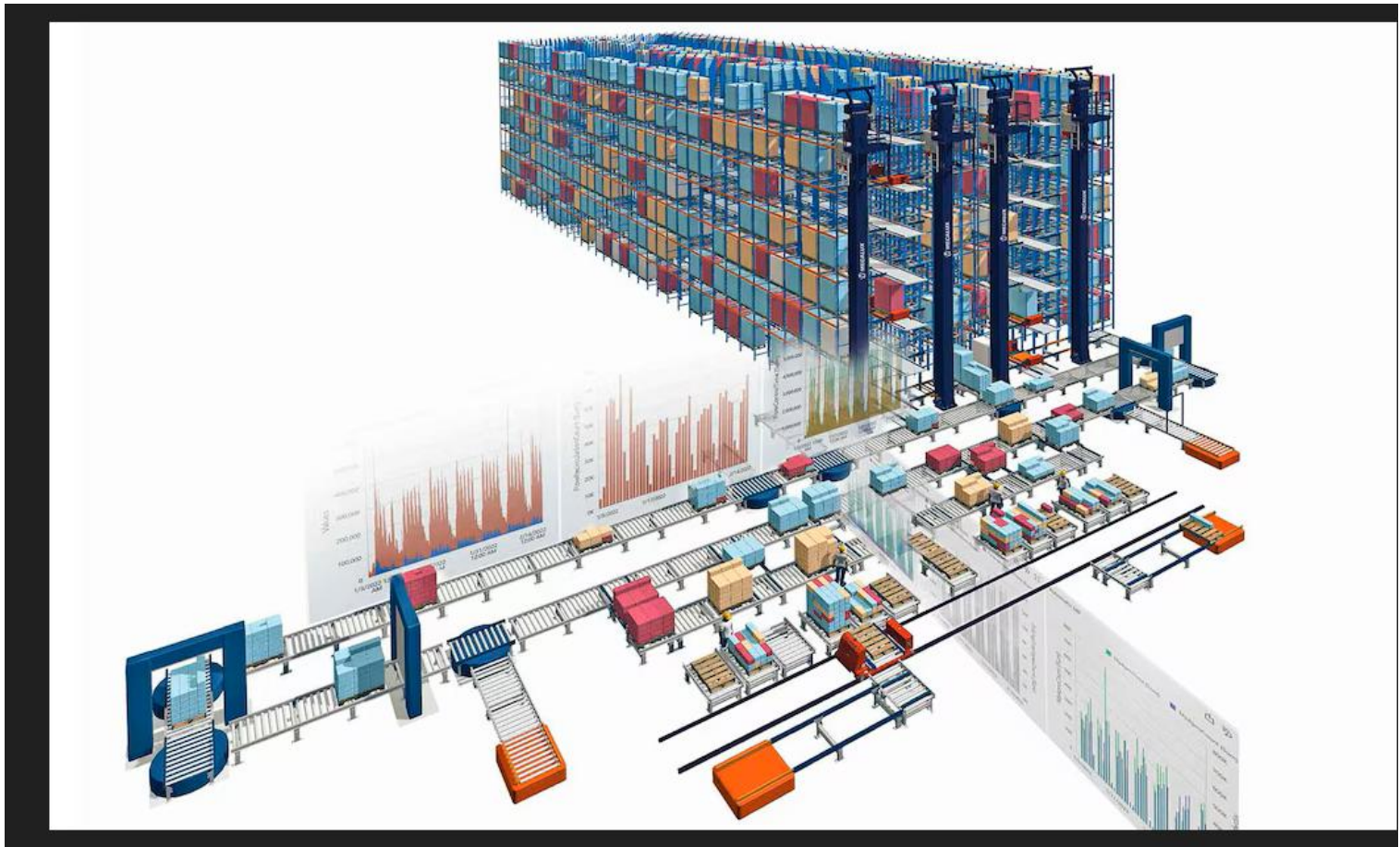


# MODULE : ALGORITHMES AVANCES



**Dr SADA ANNE**

# Chapitre 2 :

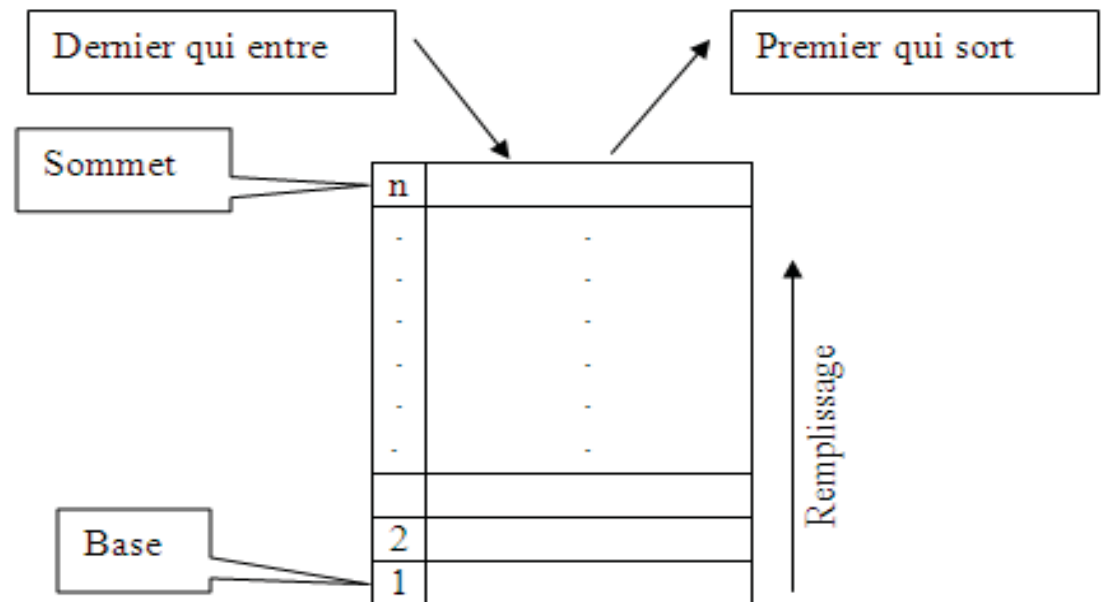
# Les Piles et Files

# Piles

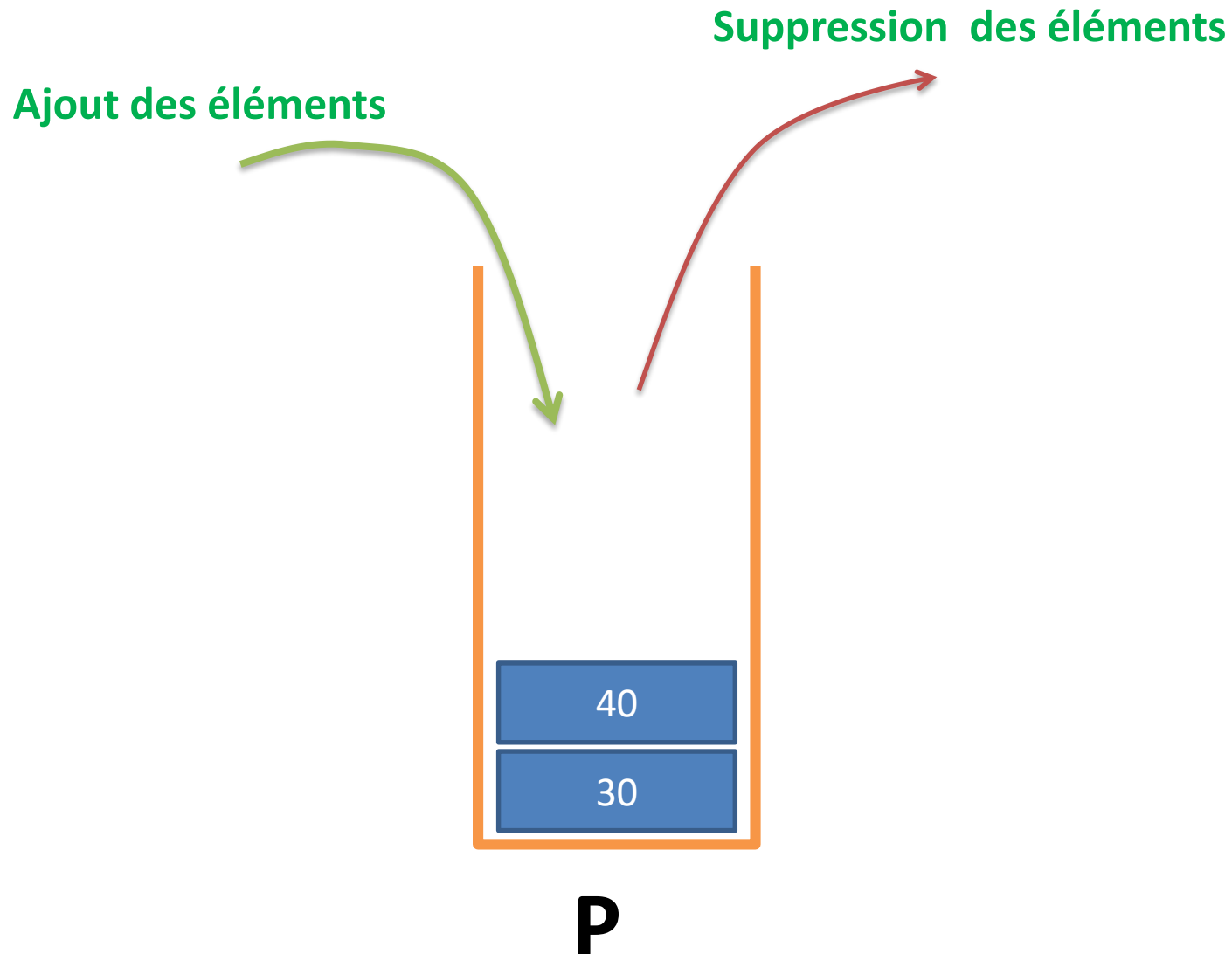
- 1. Définition**
- 2. Piles statiques**
- 3. Piles dynamiques**

# 1. Définition

- Une pile est une liste d'éléments où les insertions et les suppressions d'éléments se font à une seule et même extrémité de la liste appelée le sommet de la pile.
- Le principe d'ajout et de retrait dans la pile s'appelle LIFO (Last In First Out) : "le dernier qui entre est le premier qui sort". Il est impossible donc d'accéder à un élément au milieu.

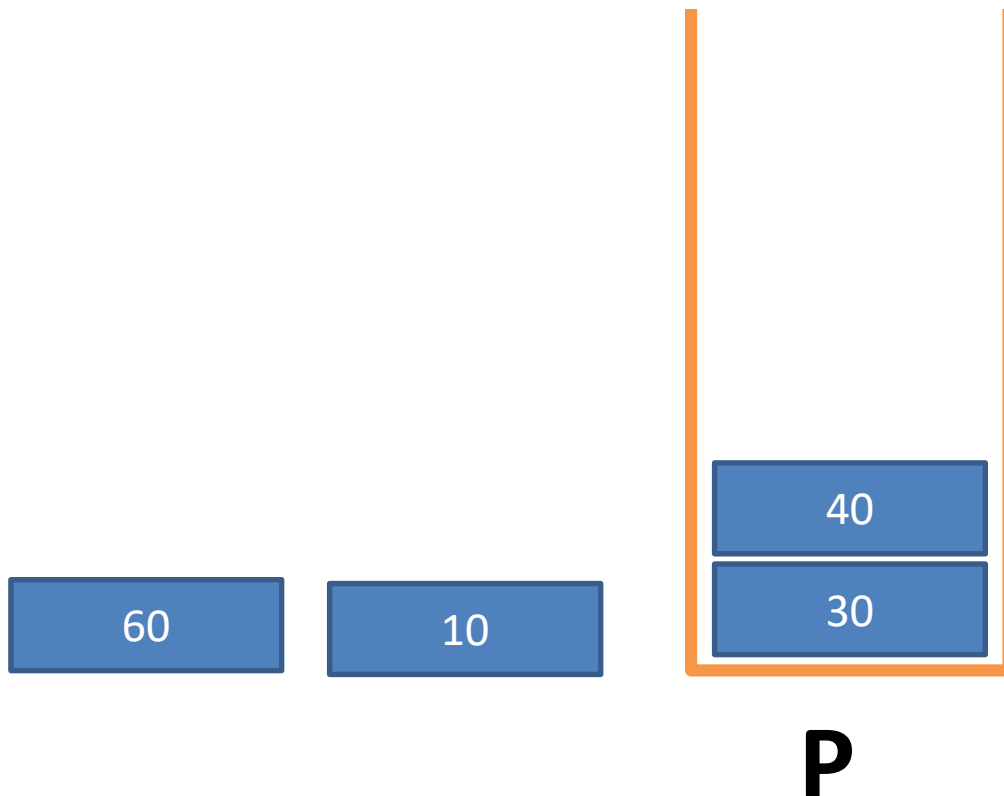


# 1. Définition



# 1. Définition

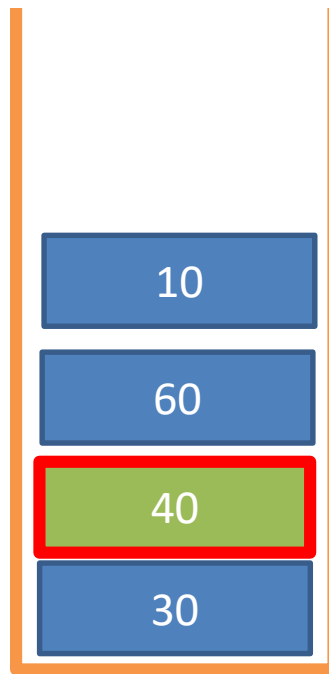
Exemple: l'ajout de deux éléments 60 et 10 dans la pile P



# 1. Définition

## Exemple 2:

- Retrait de la valeur 40 de la pile P
- Il faut tout d'abord retirer 10 et 16 et ensuite 40



**P**

# 1. Types

Selon le type d'implémentation, on distingue deux types de Piles:

- **Pile statique (contiguë):** Implémentation par un tableau.
  - La même implémentation d'une liste statique
- **Pile dynamique:** Implémentation par une liste chaînée
  - La même implémentation d'une liste chaînée



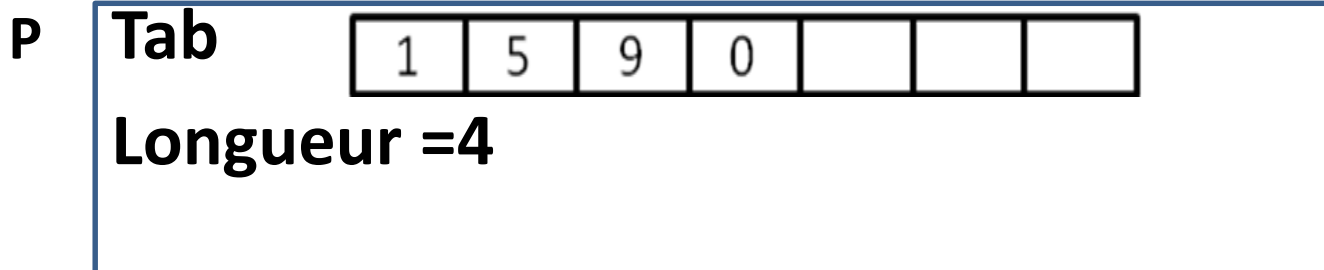
# 1. Piles statiques

Implémentation par un tableau et dans ce cas :

- la capacité de la pile est limitée par la taille du tableau.
- **L'ajout à la pile se fait dans le sens croissant des indices, tandis que le retrait se fait dans le sens inverse.**

## Exemple :

Soit la pile statique P suivante avec un tableau 7 élément (taille fixe) et une variable longueur contient la valeur 4 qui est le nombre des éléments existe dans la liste.



# 4. Pile statique

## Définition de type Pile

### Type Structure Pile

début

Tab: Tableau[MAX] d'Éléments ;

longueur : Entier ;// garde le nombre d'éléments stockés dans la  
Pile

Fin

### Exemple: Pile d'entiers

#### Type Structure **Pile**

début

Tab: Tableau[1000] d'entiers ;

longueur : Entier;

Fin

# 1. Piles statiques

## Opérations usuelles sur les piles statiques

**La procédure initialiser** : initialise la variable longueur d'une pile à la valeur 0. Elle crée donc une pile vide.

**Procédure Initialiser(var P: Pile)**

Début

    P.longueur  $\leftarrow$  0;

Fin

**La fonction est\_vide** : une fonction booléen qui reçoit une pile en entrée et retourne vrai si la pile est vide (Longueur = 0) et faux sinon.

**Fonction Est\_vide(P: Liste) : Booléen**

Début

    Retourne (P.Longueur == 0);

Fin

# 1. Piles statiques

**La fonction est pleine:** teste si une pile est pleine ou non. Elle retourne vrai si la longueur de la pile égal sa taille maximale (longueur = max);

**Fonction Est\_pleine (P : Pile) : Booléen**

Début

    Retourne (P.longueur=Max);

Fin

**La fonction sommet :** retourne l'élément en somment de la pile (le dernier élément empilé)

**Fonction sommet (P: Pile) entier**

Debut

    retourner(P.Tab[P.longueur]);

fin

# 1. Piles statiques

**La procédure empiler** : Si la Pile est non pleine, ajoute un élément en sommet de la pile et incrémente la variable longueur de 1 sinon elle ne fait rien.

**Procédure Empiler(P : Pile, x: Élémente)**

Début

Si !est\_plein(P) alors

    P.Tab [P.Longueur+1]  $\leftarrow$  x

    P.Longueur  $\leftarrow$  P.Longueur+1

Finsi

Fin

# 1. Piles statiques

**La procédure dépiler** : si la pile est non vide, elle décrémente la variable longueur de 1 sinon elle ne fait rien.

Procédure dépiler (var P: Pile)

début

    si !est\_vide(P) faire

        P.Longueur  $\leftarrow$  P.Longueur-1;

    finsi

fin

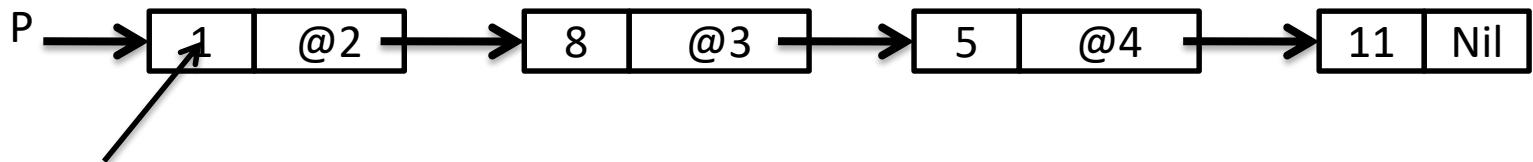
# 1. Piles statiques

**La fonction `taille`:** retourne le nombre d'élément déjà empilés dans la pile.

```
fonction taille (P: Pile): entier  
debut  
    retourner P.Longueur;  
fin
```

# 3. Piles dynamiques

C'est une Liste chaînée où l'empilement et le dépilement se font seulement à la tête de la liste.



Sommet

## Définition de type liste

Type

Structure Maillon

Ele : **typeq**; // **typeq** désigne un type quelconque (int, float, personne, étudiant ...etc.

suivant: \* **Maillon**;

fin

type Pile : \* Maillon;



# 3. Piles dynamiques

**Opérations primitives:** Les opérations usuelles sur les piles dynamiques sont les suivantes :

**Est\_vide** : retourne vrai si la Pile est vide ( $P = \text{NULL}$ ) sinon retourne faux.

Fonction Est\_vide( $P : \text{Pile}$ ) : Booléen

Début

    Si ( $P = \text{Nil}$ ) alors

        Retourner ( vrai);

    Sinon

        Retourner (faux);

    Finsi

Fin

# 3. Piles dynamiques

**Sommet** (P: Pile) : retourne l'élément qui existe dans l'entête de la Pile (Le dernier élément empilé).

**Fonction Sommet (P : Pile): type**

**Début**

**Retourner (P -> ele);**

**Fin**

# 3. Piles dynamiques

**Empiler:** empile un élément en sommet de la pile. cette fonction est équivalent à Ajouter pour une liste chaînée.

**Procedure Empiler(var P: Pile, x : typeq)**

**Q: Pile**

**Début**

**Q  $\leftarrow$  creer\_maillon (x);**

**Q -> suivant  $\leftarrow$  P;**

**P  $\leftarrow$  Q;**

**Fi n**

# 3. Piles dynamiques

**Dépiler:** retourne la pile sans son sommet.

**Procédure dépiler (var P : Pile)**

**Q : pile**

**debut**

**Si ( ! est\_vide (p)) alors**

**$Q \leftarrow P$  ;**

**$P \leftarrow p \rightarrow \text{suivant}$ ;**

**Libérer(Q) ;**

**Finsi**

**Fin**

# 1. Piles dynamiques

**La fonction taille:** retourne le nombre d'élément déjà empilés dans la pile.

Fonction Taille (P : Pile) :entier

    Courant : pile

    Nb : entier ;

Début

    Courant  $\leftarrow$  P ; Nb  $\leftarrow$  0 ;

Tantque (courant  $\neq$  NULL)

    Nb  $\leftarrow$  Nb+1 ;

    Courant  $\leftarrow$  Courant -> suivant ;

Fintantque

    Retourner n ;

Fin

# 4. Exemples d'utilisation des piles

## 1. Évaluation des expressions post-fixées

- Pour l'évaluation des expressions arithmétiques ou logiques, les langages de programmation utilisent généralement les représentations préfixée et postfixée.
- Dans la représentation postfixée, on représente l'expression par une nouvelle, où les opérations viennent toujours après les opérandes.
- Exemple
  - ✓ L'expression  $(2 + 3) * 6$  est exprimée, en postfixé, comme suit  $2\ 3+ 6*$
  - ✓ L'expression  $(a + (b * c))/(c-d)$  est exprimée, en postfixé, comme suit :  $a\ b\ c\ *\ +\ cd\ -\ /\$

## 4. Exemples d'utilisation des piles

- Pour l'évaluation des expressions postfixée, les langages de programmation utilisent le type Pile et ses primitives en parcourant l'expression de gauche à droite.

## 4. Exemples d'utilisation des piles

**Exemple** : L'algorithme suivant permettant d'évaluer l'expression arithmétique suivante en utilisant une pile comme pour un langage de programmation.  $(2 + 3) * 6$  ou  $2 3 + 6 *$  en postfixé.

$P \leftarrow \text{NiL};$

Empiler (P, 2); Empiler (P, 3);

$b \leftarrow \text{Sommet (P)}; \text{Dépiler (P)};$

$a \leftarrow \text{Sommet (P)}; \text{Dépiler (P)};$

Empiler (P,  $a+b$ );

Empiler (P, 6);

$b \leftarrow \text{Sommet (p)}; \text{Dépiler (p)};$

$a \leftarrow \text{Sommet (p)}; \text{Dépiler (p)};$

Empiler (p,  $a * b$ ) ;

Ecrire (Sommet (p)) ;

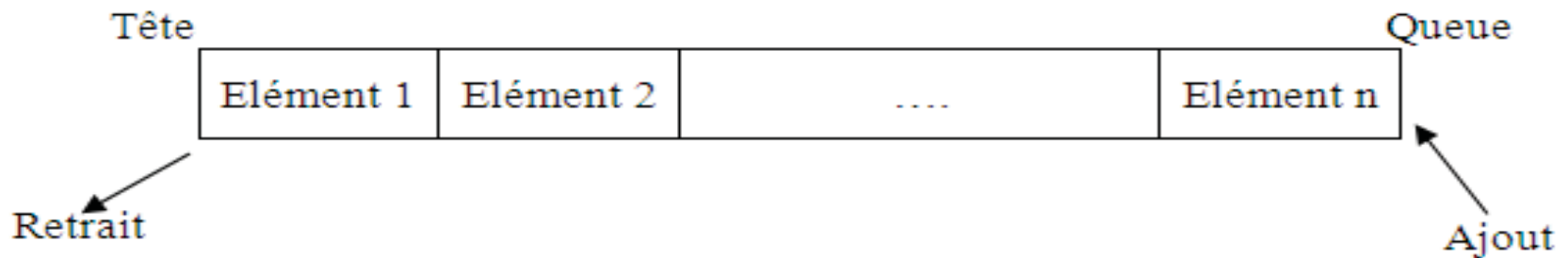


# Files

- 1. Définition**
- 2. Files statiques**
- 3. Files dynamiques**

# 1. Définition

- La file d'attente est une structure qui permet de stocker des éléments dans un ordre donné et de les retirer dans le même ordre, c'est à dire selon le protocole FIFO 'first in first out'.
- «On ajoute toujours un élément en queue et on retire celui qui est en tête»



# 1. Définition

Selon le type d'implémentation, on distingue deux types de files:

- **File statique (contiguë):** Implémentation par un tableau.
- **File dynamique:** Implémentation par une liste chaînée.

## 2. Files statiques

- Implémentation par un tableau.
- La définition du type **File** (statique) est comme suit :

**Type Structure File**

**début**

**Tab: Tableau[MAX] d'Éléments ;**

**Tête : entier ; // indice du premier de la File.**

**Queue: entier ; // indice du dernier élément inséré dans la File.**

**Fin**

**F**

**Tab:**

10

20

14

7

**Tete: 1**

**Queue: 4**

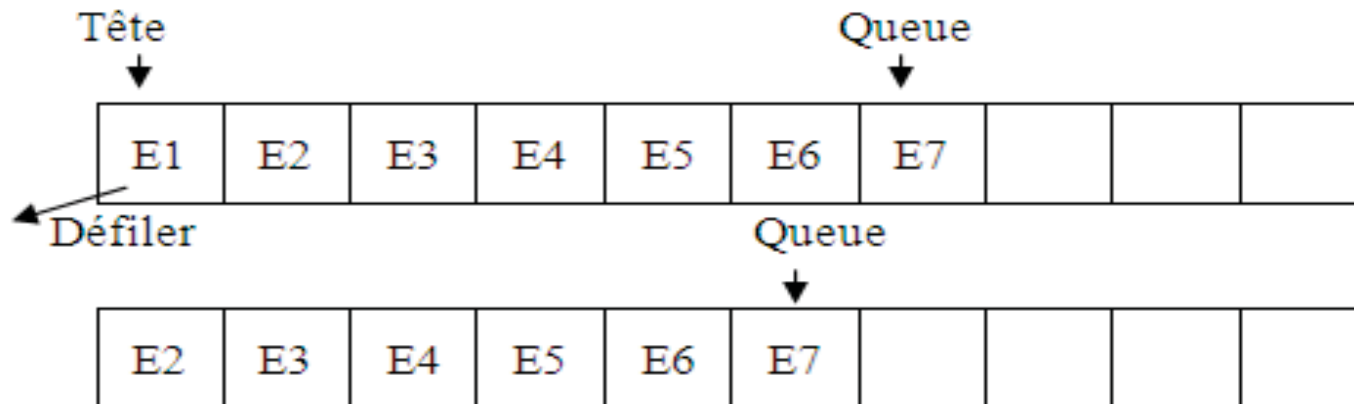
## 2. Files statiques

- L'implémentation de files statiques peut être réalisée avec deux manière :
  - **Par décalage:** en utilisant un tableau avec une tête fixe, toujours à 1, et une queue variable.
  - Elle peut être aussi réalisée par flot en utilisant un tableau circulaire où la tête et la queue sont toutes les deux variables.

# 2. Files statiques

## Implémentation par décalage

- Tête fixe et queue variable.

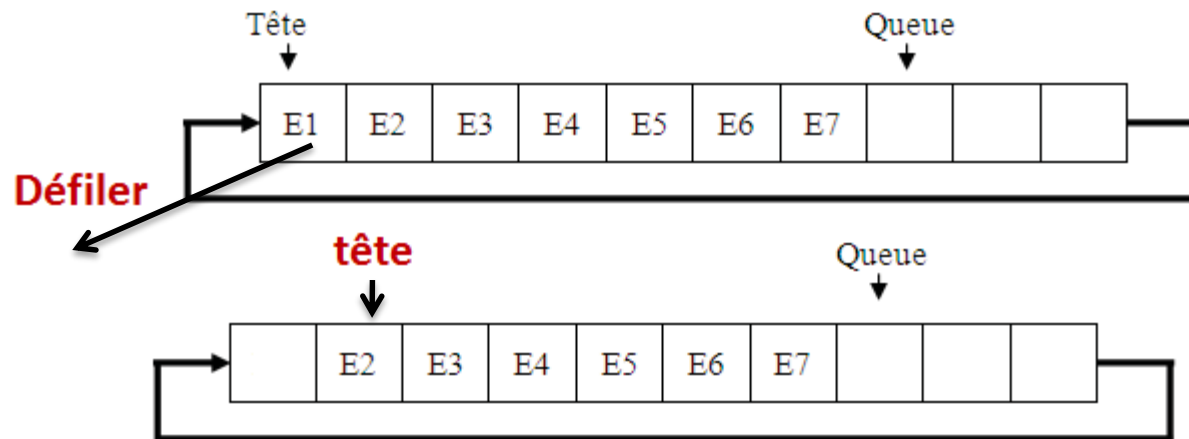


- La file est vide si  $Queue = 0$
- La file est pleine si  $Queue = Max$
- **Elle souffre du problème de décalage à chaque défilement.**

# 2. Files statiques

## Implémentation par un tableau circulaire

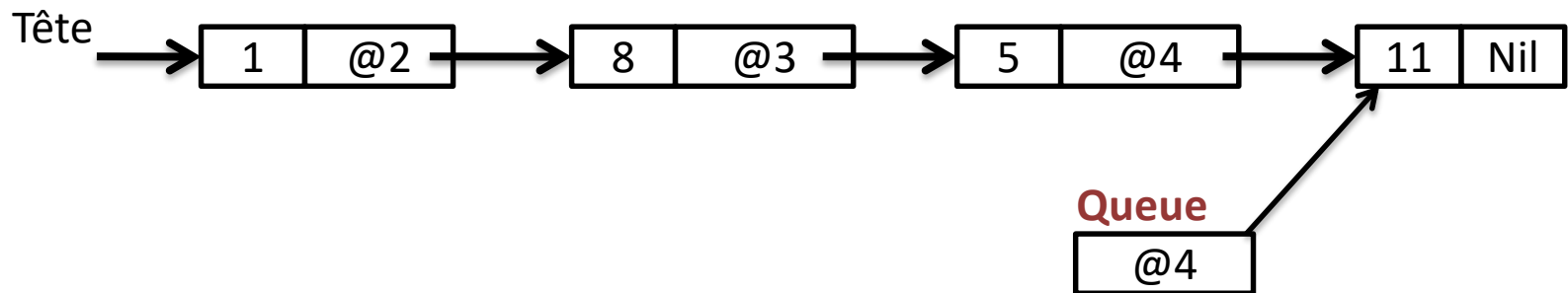
- Tête variable et queue variable).



- La file est vide si  $Tête = Queue$
- La file est pleine si  $(Queue + 1) \bmod Max = Tête$

# 3. Files dynamiques

C'est une Liste chaînée où le défilement se fait seulement à la tête et l'enfilement se fait seulement à la queue de la liste.





# 3. Files dynamiques

## Définition de type File dynamique

La définition du type **File** (dynamique) est comme suit :

### Type Structure Maillon

Ele : **typeq**;

suivant: **\* Maillon**;

fin

### Type Structure File

Début

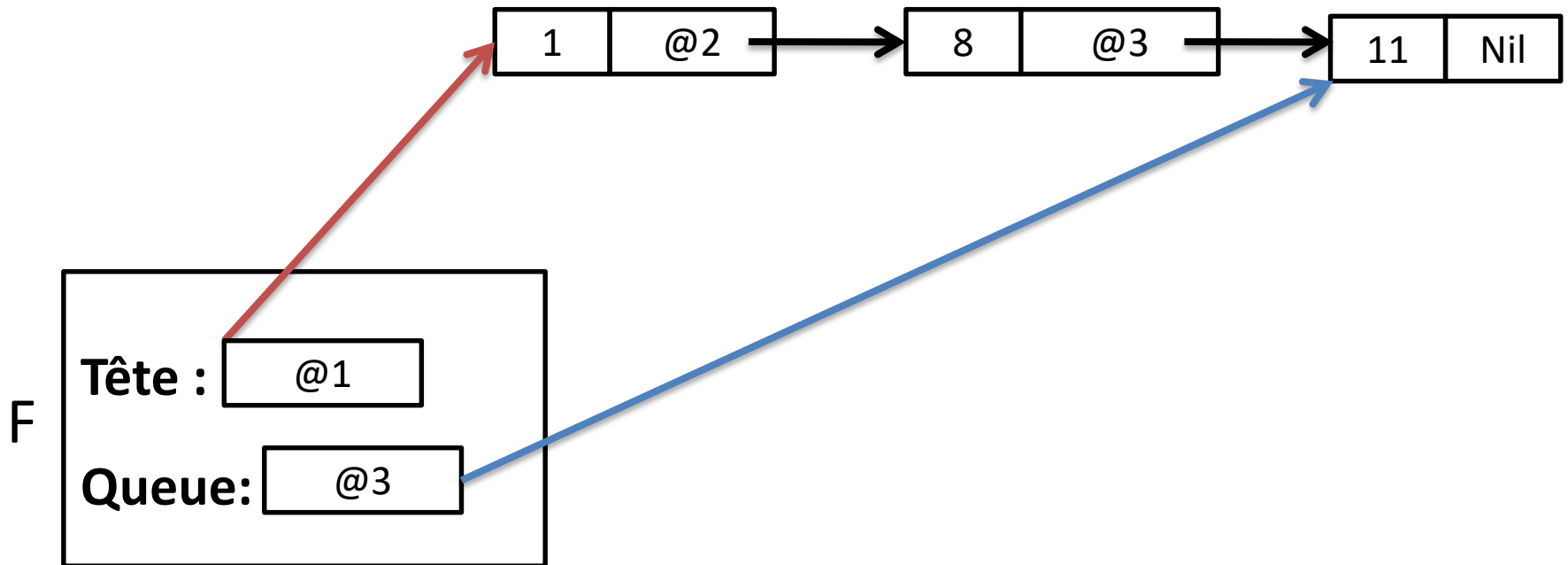
**Tête : \*Maillon ;** // Gade l'adresse de la tête de la file.

**Queue: \*Maillon;** // Gade l'adresse du dernier élément de la file.

Fin

# 3. Files dynamiques

La structure d'une file dynamique



# 3. Files dynamiques (Opérations primitives)

1. **Initialiser** : initialise les variables Tête et Queue de la file à Nil.

Procédure Initialiser(var F: Pile)

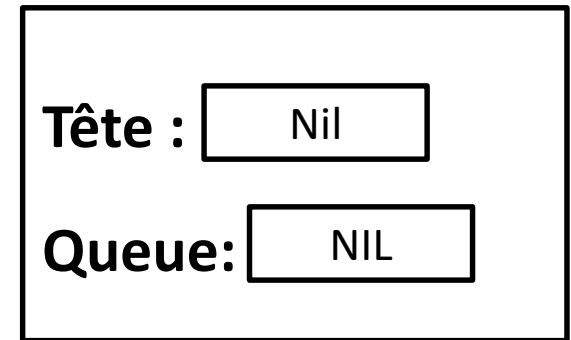
Début

F.Tête ← Nil;

F.Queue ← Nil;

Fin

F



2. **Est\_vide**: retourne vrai si la file est vide sinon retourne faux.

Fonction est\_vide (F: File): booléen

début

Retourne (F.Tête =NULL);

fin

# 3. Files dynamiques

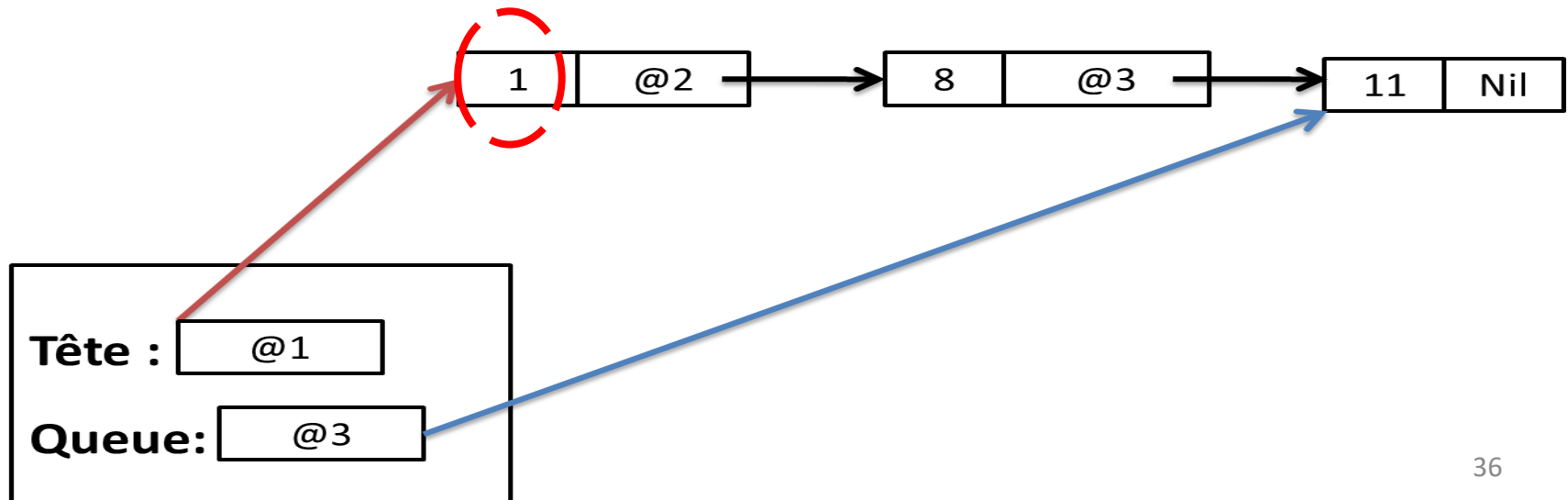
**3. Tête** : retourne l'élément qui existe dans l'entête de la File (Le premier élément enfilé).

**Fonction tête (F: File) : entier  
début**

Retourne (**F.Tête -> Ele**);

**fin**

**F.Tête -> Ele = 1**



# 3. Files dynamiques

**Enfiler** : ajoute un élément en queue de la file.

Procédure enfiler (var F: File, x: typeq)

P: \*maillon

Début

P  $\leftarrow$  créer\_maillon (x);

Si (est\_vide (F)) alors

F.tete  $\leftarrow$  P;

F.Queue  $\leftarrow$  P;

Sinon

F. Queue -> suivant  $\leftarrow$  P;

F.Queue  $\leftarrow$  P;

Finsi

Fin

# 3. Files dynamiques

**Défiler** : supprime le premier élément (la tête) de la file.

Procédure défiler (var F: File)

    P: \*maillon

Début

    Si( ! est\_vide (f))

        P  $\leftarrow$  F.tête;

        F.tete  $\leftarrow$  f.tete ->suivant;

        Libérer (p);

        Si (F.tete =Nil)

            F.Queue  $\leftarrow$  Nil;

        Finsi

    Finsi

Fin

# 3. Files dynamiques

- **La fonction taille:** retourne le nombre d'éléments déjà enfilés dans la pile.

**Fonction Taille (File : Pile) :entier**

Courant : \*Maillon

Nb : entier ;

**Début**

Courant  $\leftarrow$  F.Tête ; Nb  $\leftarrow$  0 ;

Tantque (courant  $\neq$  Nil)

    Nb  $\leftarrow$  Nb +1;

    courant  $\leftarrow$  courant->suivant ;

Fintantque

Retourner Nb ;

**fin**